

ModelSmith - Problem Definition & Solution

1. Problem Definition

Modern AI models are increasingly large and computationally expensive. A trained model is rarely ready for deployment: engineers must determine whether it fits the target hardware, meets latency and memory limits, and preserves acceptable accuracy.

Current Problems

Problem	Impact
Manual optimization	Engineers repeatedly try quantization, pruning, graph optimization and other techniques.
Trial-and-error benchmarking	Every candidate often has to be executed and benchmarked before its value is known.
Hardware dependency	A technique suitable for a GPU may be unsuitable for CPU, edge, mobile or another runtime.
Multiple trade-offs	Model size, latency, memory, throughput and accuracy can improve or worsen together.
Fragmented tooling	Analysis, optimization, benchmarking and deployment tools are spread across different frameworks.
High expertise requirement	Selecting the right optimization strategy requires specialized ML and deployment knowledge.

2. Problem Statement

There is a need for an intelligent system that can analyze an AI model, understand deployment requirements and target hardware, recommend suitable optimization strategies, predict their trade-offs, and guide or execute the selected optimization workflow.

3. Proposed Solution - ModelSmith

ModelSmith is an intelligent AI model optimization and deployment platform. It acts as a decision-making and orchestration layer between model development and deployment. Instead of relying entirely on repeated manual experimentation, it analyzes the model and generates ranked optimization plans before execution.

Core Workflow

Stage	Main Function	Output
1. Upload	Validate and store the supported model.	Model + metadata
2. Analyze	Inspect graph, layers, parameters, FLOPs, tensors and memory.	Analysis report
3. Profile	Estimate latency, memory and computational hotspots.	Performance profile
4. Plan	Use model features, goals, hardware and optimization knowledge to generate candidates.	Ranked plans
5. Predict	Estimate size, latency, memory, throughput and accuracy impact.	Prediction report
6. Compare	Show plans and their trade-offs so the user can select one.	Selected plan
7. Execute	Run the selected optimization pipeline using supported tools.	Optimized model
8. Validate/Benchmark	Compare original and optimized models.	Benchmark report
9. Export	Prepare deployment-ready model and configuration artifacts.	Artifact package

4. Intelligent Optimization Engine

- Model features: architecture, operators, parameters, FLOPs, memory and runtime characteristics.
- Deployment requirements: target objective such as lowest latency, lowest memory, smallest size, edge deployment or balanced optimization.
- Hardware profile: CPU/GPU/edge/mobile target, available memory, runtime and supported precision.
- Knowledge base: relationships between model types, optimization techniques, hardware and expected outcomes.
- Planner: generates candidates, removes incompatible strategies, ranks valid plans and explains recommendations.
- Prediction engine: estimates expected performance before expensive optimization execution.

5. Expected Benefits

Benefit	Expected Result
Less experimentation	Avoid unnecessary optimization attempts by predicting likely outcomes first.
Faster deployment decisions	Compare multiple strategies in one workflow.
Hardware-aware optimization	Recommendations match the intended deployment environment.
Explainable decisions	Users understand why a strategy is recommended and what trade-offs it has.
Reproducibility	Model, plan, hardware, predictions and execution results remain linked.

6. Scope

In scope: model upload, analysis, profiling, optimization planning, performance prediction, plan comparison and reporting. Optimization execution, benchmarking and deployment export are planned extensions. Training models, dataset engineering and replacing inference runtimes are outside the primary scope.